

MigsFlow Web Development Handbook

Coding Standards, File Structure, Performance Rules, and AI Agent Instructions

Purpose

This handbook defines the default way MigsFlow websites should be structured and coded. It is written so it can be handed directly to an AI coding agent, collaborator, or future developer. Unless a project explicitly requires an exception, these rules should be treated as the standard.

Core philosophy: keep code organized, reusable, lightweight, locally controlled, responsive, and optimized for page speed.

MigsFlow Web Design

Version 1.0

1. Project Structure

Every project should begin with a predictable structure. The homepage file remains at the project root, while supporting files are organized by language or asset type.

Required baseline structure

```
project-root/
|-- index.html
|-- html/
|   |-- about.html
|   |-- contact.html
|   `-- other-pages.html
|-- css/
|   |-- global.css
|   |-- styles.css
|   |-- about.css
|   |-- contact.css
|   `-- critical.css (when needed)
|-- javascript/
|   |-- global.js
|   |-- script.js
|   `-- page-specific.js
|-- images/
|   |-- image.webp
|   |-- image.jpg
|   |-- image-m.webp
|   `-- image-m.jpg
|-- fonts/
`-- assets/ (optional)
```

- index.html must remain at the root.
- Every additional HTML page belongs in the html/ folder.
- All CSS belongs in css/. All JavaScript belongs in javascript/.
- All self-hosted font files belong in fonts/.
- Images belong in images/ and should use consistent, descriptive file names.

2. Separation of Concerns

Hard rule

Never write CSS or JavaScript directly inside an HTML file. HTML, CSS, and JavaScript must remain in their own files.

- HTML is responsible for semantic page structure and content.
- CSS is responsible for presentation, layout, responsive behavior, and visual states.
- JavaScript is responsible for interactivity and behavior.
- Do not use inline style attributes, embedded `<style>` blocks, or embedded JavaScript blocks as a normal coding practice.
- If an external integration absolutely requires an inline snippet, treat it as a documented project exception rather than the default.

3. Global vs. Page-Specific Code

Reusable code should be written once and shared. Page-specific files should contain only what is unique to that page.

global.css

- Navigation and mobile navigation styles
- Footer styles
- Primary and secondary buttons
- Font declarations and shared typography
- Shared containers, spacing utilities, repeated cards, forms, or common components
- Site-wide colors, variables, resets, and other reusable standards

Page-specific CSS

- styles.css is the homepage stylesheet.
- about.css, contact.css, and similar files contain only styles unique to those pages.
- Before adding a rule to a page stylesheet, check whether the same pattern already exists or is likely to repeat elsewhere. If yes, place it in global.css instead.

JavaScript follows the same rule

- Shared behaviors such as navigation toggles or reusable site-wide interactions belong in global.js.
- Homepage-only behavior belongs in script.js or another clearly named homepage file.
- Page-specific behavior belongs in a clearly named page-specific JavaScript file.
- Do not duplicate the same function across multiple page files when it can be centralized.

4. Image Standards and Performance

Hard rule

Every raster image added to the project should have a WebP version as the primary optimized format and a JPG fallback when JPG is appropriate for the source image. Mobile-specific crops should also be created when they materially reduce file size or improve framing.

Responsive images

- Use the HTML `<picture>` element when serving different image sources or formats.
- Create mobile and desktop versions when useful. Use a -m suffix for mobile crops, such as kitchen-m.webp.
- Crop images close to their actual display dimensions. A practical high-density target from the performance reference is approximately 2x the displayed width and height.
- Compress images before adding them to production.
- As a performance target, keep ordinary mobile images roughly in the 20-40 KB range when practical. Background images should generally remain below 100 KB.

- Use descriptive alt text for meaningful images and explicit width and height attributes to help prevent layout shift.

Loading behavior

- Every image should use `decoding="async"` unless there is a specific reason not to.
- Images below the fold should use native `loading="lazy"`.
- Never lazy load above-the-fold or critical landing-page images.
- Preload critical above-the-fold images such as the hero image or logo when testing shows it is beneficial.
- For large visual backgrounds, prefer a responsive picture element positioned as a background layer when this allows better mobile sources, lazy loading, and async decoding.

5. Font Standards

Hard rule

Never pull production fonts from Google Fonts or another font CDN. MigsFlow fonts are self-hosted and stored in the project's `fonts/` folder.

- Create a `fonts/` folder for every project.
- Use locally hosted font files supplied or approved for the project.
- Declare shared `@font-face` rules near the top of `global.css`.
- Use `font-display: swap` for locally hosted fonts.
- Load only the font weights and styles the site actually needs. Avoid unnecessary font variants.
- Preload only font files required above the fold. Do not preload every font by default.
- Choose a sensible system fallback font that closely matches the primary font to reduce content layout shift.

6. JavaScript and Third-Party Code

- Keep JavaScript lightweight and use native browser/JavaScript capabilities when practical.
- Defer nonessential external JavaScript files so they do not block initial rendering.
- Avoid jQuery unless a project has a specific dependency that genuinely requires it.
- Avoid Font Awesome as a default icon solution. Prefer optimized SVG assets.
- Limit third-party scripts and CDNs because they can add page weight, render blocking, privacy dependencies, and additional failure points.
- When adding analytics or another third-party script, evaluate its performance impact and load it as late as practical without breaking its intended function.

7. CSS Performance Rules

- Keep `global.css` focused on genuinely reusable code and avoid unnecessary duplication.

- Minify production CSS and JavaScript during deployment when the hosting/build process supports it.
- If a large stylesheet is render blocking and performance testing shows a meaningful benefit, create critical.css containing only above-the-fold styles.
- When deferring noncritical CSS, include a <noscript> fallback so the page remains styled when JavaScript is unavailable.
- Do not add performance techniques mechanically. Verify that they improve the actual site and do not harm maintainability or accessibility.

8. SVG, Accessibility, and Code Quality

- Prefer SVG for logos, interface icons, social icons, phone/email icons, and other vector graphics when appropriate.
- Optimize SVG files before production use.
- Use semantic HTML elements whenever possible.
- Maintain a logical heading hierarchy and accessible labels for interactive controls.
- All interactive elements must be keyboard usable and have visible focus behavior.
- Use descriptive class and file names. Avoid vague names such as box1, thing, stuff, or temp.
- Remove dead CSS, unused JavaScript, commented-out experiments, and duplicate code before launch.
- Keep code readable and consistently formatted even when production assets are later minified.

9. MigsFlow Build Workflow

- 1. Create structure: establish root index.html plus html/, css/, javascript/, images/, and fonts/.
- 2. Establish global system: define fonts, colors, resets, shared typography, buttons, navigation, footer, and reusable components in global.css.
- 3. Build semantic HTML: create page structure without mixing in CSS or JavaScript.
- 4. Add page-specific styling: keep only unique styles in that page's stylesheet.
- 5. Add behavior: centralize reusable JavaScript and isolate page-specific interactions.
- 6. Optimize assets: crop, compress, create WebP/JPG variants, add mobile crops, optimize SVGs, and self-host fonts.
- 7. Apply loading strategy: lazy load below-the-fold images, async decode images, preload only critical assets, and defer nonessential JavaScript.
- 8. Test: check responsive layouts, accessibility, broken links, console errors, and performance.
- 9. Clean production code: remove duplicates and unused code, then minify production CSS/JS when supported.
- 10. Final performance pass: use the CodeStitch Page Speed Handbook as the reference checklist and validate changes with real performance testing.

10. Instructions for an AI Coding Agent

AI operating instruction

When working on a MigsFlow website, follow this handbook as the default coding standard. Do not silently change the architecture, introduce frameworks, add external font services, duplicate reusable code, or place CSS/JavaScript inside HTML. If a requested feature conflicts with a hard rule, explain the conflict before implementing an exception.

Before writing code, the agent should:

- Inspect the existing project structure and reuse established global styles and functions.
- Determine whether new CSS or JavaScript is global/reusable or unique to the current page.
- Avoid introducing a dependency when the feature can reasonably be built with HTML, CSS, or vanilla JavaScript.
- Preserve existing naming conventions and responsive behavior.
- Use locally hosted fonts only.
- Use optimized image markup and appropriate loading attributes.
- Keep changes focused. Do not rewrite unrelated code unless the user asks for a refactor.
- When returning code changes, clearly identify which file each block belongs in.

Definition of done

- No inline CSS or JavaScript.
- No unnecessary duplicated styles or functions.
- No Google Fonts CDN.
- Responsive image strategy is in place.
- Below-the-fold images are lazy loaded; critical images are not.
- Images use async decoding and explicit dimensions where appropriate.
- Fonts are local and only necessary weights are loaded.
- Nonessential JavaScript is deferred.
- Reusable code lives in global files.
- Page-specific files contain only page-specific code.
- The site has been checked for responsiveness, accessibility basics, console errors, and performance regressions.

11. Performance Reference Notes

The attached CodeStitch material emphasizes responsive picture elements, display-size cropping, compression and WebP conversion, native lazy loading below the fold, async image decoding, critical-image preloading, local font hosting, font preloading, CSS/JS minification, SVG use, deferred nonessential JavaScript, removal of unnecessary third-party scripts, and critical CSS when needed.

Primary performance reference: CodeStitch Page Speed Handbook. MigsFlow's own hard rules in this handbook take priority where the two differ.

Important distinction

The CodeStitch material is used here as a performance reference, not as permission to copy every implementation blindly. MigsFlow standards should remain maintainable, accessible, and project-appropriate. Test optimizations instead of assuming every technique is necessary on every page.

12. Mobile-First CSS and Responsive Design

This section is part of the MigsFlow coding standard and should be followed by developers and AI coding agents on every project.

Hard rule: Mobile-first CSS

All MigsFlow CSS must be written mobile first. Base CSS should represent the mobile layout and the smallest practical viewport. Media queries are then used to progressively enhance or adjust the design for larger screens. Do not build the desktop layout first and use media queries to shrink it down for mobile.

- **Start with mobile.** Write the default CSS for phones and smaller viewports first.
- **Use min-width media queries for larger devices.** Add layout enhancements as screen space increases rather than using max-width queries to undo desktop styling.
- **Responsive on every device.** Mobile-first does not mean mobile-only. Every website must remain functional, readable, visually balanced, and responsive across phones, tablets, laptops, desktops, and larger screens.
- **Avoid device-specific assumptions.** Layouts should adapt fluidly between breakpoints rather than only looking correct at a few exact screen sizes.
- **Use flexible sizing where appropriate.** Prefer responsive techniques such as percentages, max-width, min(), max(), clamp(), Flexbox, and CSS Grid instead of unnecessary fixed dimensions.
- **Test multiple viewport widths.** Check the design between standard breakpoints to catch overflow, awkward wrapping, oversized spacing, and layout jumps.
- **Protect usability.** Navigation, buttons, forms, images, text, and interactive elements must remain usable and accessible at all supported viewport sizes.
- **Keep media queries purposeful.** Only add a breakpoint when the layout actually needs to change. Do not create media queries simply because a specific device size exists.

AI agent instruction: When generating or editing MigsFlow CSS, assume mobile-first architecture unless explicitly told otherwise. Write the smallest-screen styles as the base rules, then use min-width media queries to enhance the layout for larger viewports. Do not sacrifice tablet, desktop, ultrawide, or intermediate-width responsiveness in the process.